



جامعة دمشق  
كلية الهندسة  
المعلوماتية  
قسم الذكاء الصناعي

## مشروع البرمجة التفرعية Ecommerce System

المهندس:

حذيفة عقيل

الطالبات:

البتول حسام صراميجو  
بتول صهيب مستو  
حلا عزو سليمان  
حنين سامي قاطوع  
سارة محمد غسان فيوم



المشروع عبارة عن متجر الكتروني تم تطويره باستخدام:

## Django, Django REST Framework, PostgreSQL

يهدف المشروع الى إدارة:

- المنتجات(Products)
  - الطلبات(Orders)
  - المستخدمين(Users)
  - المخزون(Stock Management)
- مع التركيز على: حماية البيانات من التضارب.

### سبب اختيار Django:

- بنية جاهزة وسريعة.
- يحوي على نظام جاهز للمستخدمين.
- للتعامل مع قاعدة البيانات بسهولة عبر الـ ORM.
- دعم المعاملات لضمان سلامة العمليات باستخدام `transaction.atomic()`.
- دعم التزامن.
- يدعم التعامل مع الطلبات المتزامنة.
- يوفر حلولاً مثل:
  - Atomic Updates باستخدام `F()`.
  - Row Locking باستخدام `select_for_update()`.
- يسهل بناء APIs بسهولة واحترافية.

### تصميم النظام (System Design):

#### :Models

1. Product: الحقول (Name, stock, price).
2. Order: الحقول (User, status [PENDING, PAID, COMPLETED, CANCELLED], created\_at).
3. OrderItem: الحقول (Order, Product, quantity).

#### APIs المنفذة:

1. المستخدمين: Register, login with JWT (Bearer token).
2. المنتجات: عرض كل المنتجات، عرض منتج واحد، اضافة منتج، تعديل منتج، حذف منتج.
3. الطلبات: \* إنشاء طلب شراء (PENDING).
  - a. دفع الطلب (PAID).
  - b. إتمام الطلب.
  - c. الغاء الطلب (CANCELLED).

PostgreSQL أقوى من SQLite في locking, transactions لأنها تدعم العمليات المتزامنة.

### الطلب الأول: حماية البيانات المشتركة من التضارب عند وصول عدة طلبات في نفس الوقت

المشكلة هي Race Condition، تحدث عندما عدة مستخدمين يطلبون نفس المنتج بنفس الوقت. مثال:

Stock = 2 ويوجد 3 طلبات بنفس الوقت، بدون حماية سوف ينجح ويقرأ جميع الطلبات بنجاح ويصبح - Stock = 1.

السبب أن العمليات تتم بشكل غير متزامن ولا يوجد قفل أو حماية على البيانات.

الحلول الممكنة:

#### 1. Row locking using Select\_for\_update ()

يقوم بقفل الصف داخل قاعدة البيانات، مشكلته أنه أبطأ ويسبب انتظار.

#### 2. Atomic Update Product.objects.filter(id = product\_id, stock\_gte = quantity).update(stock=F('stock') - quantity))

وهو الحل المستخدم في المشروع، تم استخدام Transaction.atomic() باستخدام transaction لضمان أن العملية كاملة تنجح أو تفشل باستخدام Atomic Update F() ، يتم التحقق من أن stock >= quantity ويتم التعديل مباشرة داخل قاعدة البيانات بدون قراءة القيمة أولاً.

**والنتيجة:** العملية تتم في خطوة واحدة لا يمكن لطلبين التعديل بنفس الوقت بشكل خاطئ يتم منع Race Condition.

السبب لاختيار هذا الحل لأنه أسرع، أبسط ، ويمنع التضارب بكفاءة عالية.

#### لاختبار هذا الطلب (اختبار التزامن):

لإرسال عدة طلبات بنفس الوقت تم استخدام سكربت والنتيجة كانت:

201 Created

201 Created

400 Not enough stock

أي أنه لم يتم بيع أكثر من الكمية المتاحة ولم يصبح المخزون سالب والنظام آمن من Race Condition.

### الطلب الثاني: إدارة الموارد والتحكم في القدرة Capacity Control:

في هذا الجزء، ركزنا على هندسة الموارد الداخلية للنظام لضمان استقراره تحت الضغط العالي دون الحاجة لانتظار المهام الطويلة.

#### ١. الية تجميع الخيوط Thread Pooling Strategy:

بدلاً من ترك المعالج يستهلك طاقته في فتح وإغلاق خيوط معالجة Threads بشكل عشوائي مع كل طلب، قمنا بحصر الموارد المتاحة للنظام:

تحديد القدرة الاستيعابية: استخدمنا `os.cpu_count()` لتحديد عدد الأنوية المتاحة في الجهاز وبناء عليه حددنا عدد العمال (Workers) وبالأستفادة من المعادلتين الآتيتين :

### المهام المقيدة بالمعالج CPU-Bound Tasks:

المعادلة:  $N + 1$

الشرح: تُستخدم للمهام التي تستهلك المعالج بالكامل. نكتفي بهذا العدد لأن زيادة الخيوط عن عدد الأنوية لن تزيد السرعة، بل ستسبب بطناً نتيجة تبديل السياق Context Switching

### المهام المقيدة بالإدخال والإخراج (I/O-Bound Tasks):

المعادلة:  $N \times (1 + W/C)$

الشرح: حيث  $W$  زمن الانتظار و  $C$  زمن المعالجة تستخدم لعمليات قاعدة البيانات والشبكة وبما أن الخيوط تكون خاملة أثناء الانتظار يمكننا مضاعفة عددها مثلاً  $5 \times N$  لاستغلال المعالج في مهام أخرى.

### فصل المهام حسب النوع:

`IO_Worker` خصصناه للمهام التي تقضي معظم وقتها في انتظار قاعدة البيانات مثل `save` و `update` `CPU_Worker` للمهام التي تحتاج عمليات حسابية.

الفائدة الهندسية: منعنا النظام من الوصول لحالة الانهيار Crash نتيجة استهلاك الذاكرة المفرط إذا وصل الـ `fork` المستخدمين في لحظة واحدة.

```
19
20 n_cores = os.cpu_count() or 1
21
22 IO_WORKERS = n_cores * 5
23 io_executor = ThreadPoolExecutor(max_workers=IO_WORKERS, thread_name_prefix="IO_Worker")
24
25 CPU_WORKERS = n_cores + 1
26 cpu_executor = ThreadPoolExecutor(max_workers=CPU_WORKERS, thread_name_prefix="CPU_Worker")
27
```

## ٢. محاكاة التحكم في الأحمال Capacity Simulation:

- الاستجابة الفورية Asynchronous Response في الـ `register` والـ `Order Creation` بمجرد التأكد من صحة البيانات (`is_valid`) يتم تفويض المهمة للـ `Executor` ليعمل في خيط منفصل وإعادة رد فوراً للمستخدم بحالة `Accepted` ٢٠٢ هذا يمنع بقاء المستخدم في حالة انتظار ويحرر خيط المعالجة الرئيسي فوراً.
- التحكم بالزمن Time/Resource Simulation أضفنا `time.sleep(2)` في العمليات مثل الدفع `process_payment` لمحاكاة استهلاك المورد لفترة زمنية Latency ورغم هذا التأخير المتعمد، يبقى النظام قادراً على استقبال طلبات جديدة لأن المهمة تعمل في الخلفية الداخلية للتطبيق دون تعطيل المسار الرئيسي.

## ٣. التخزين المؤقت الذكي Internal Caching:

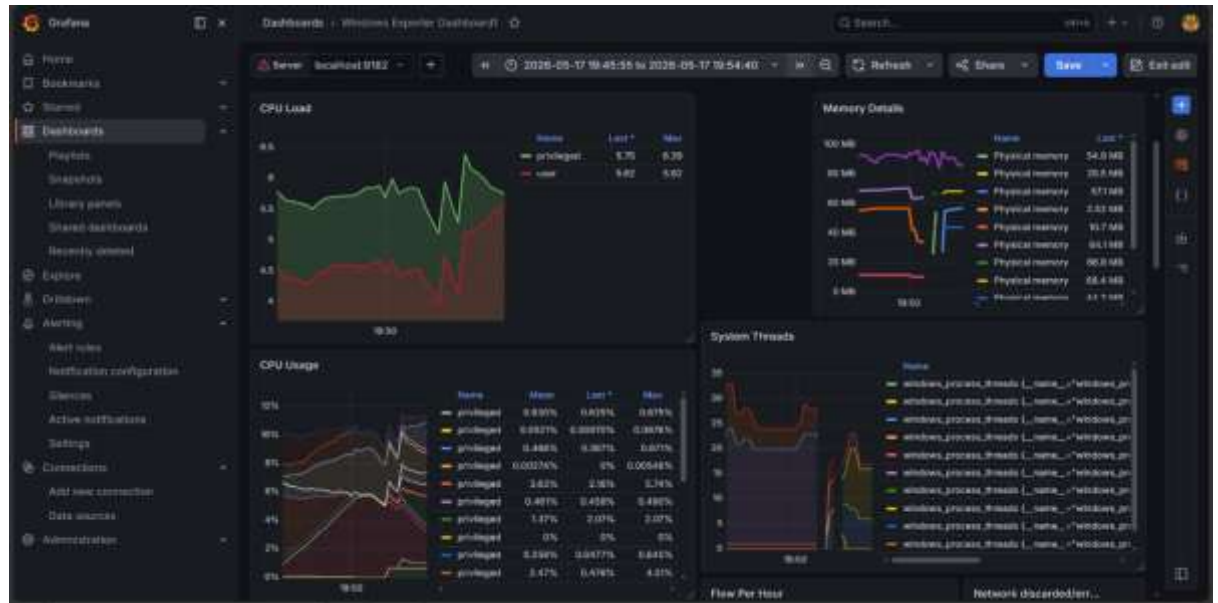
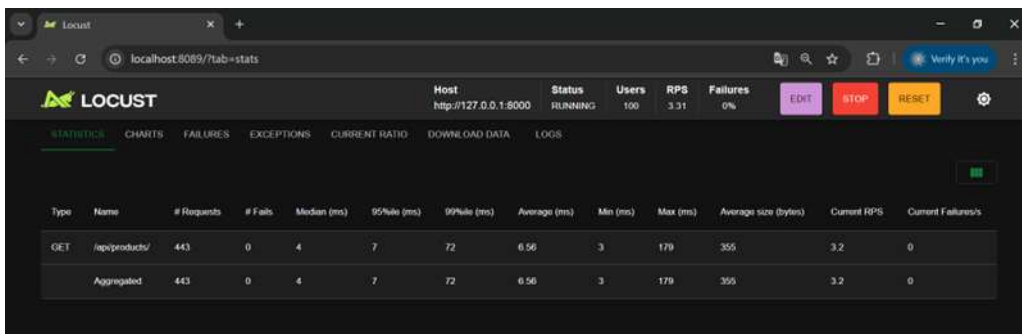
- تقليل العبء Database Offloading استخدمنا الكاش في `ProductListCreateAPIView` لتقليل الضغط على المورد الأكثر استهلاكاً وهو (قاعدة البيانات) مما يسرع زمن الاستجابة بشكل ملحوظ.....
- تحديث الموارد Background Refresh قمنا بتطبيق فكرة التحديث الخفي حيث يحصل المستخدم على البيانات من الكاش فوراً بينما يقوم خيط معالجة منفصل بتحديث الكاش من قاعدة البيانات في الخلفية هذا يضمن بقاء المورد متاحاً دائماً High Availability ويقلل من حدوث الاختناقات عند تحديث البيانات الضخمة.
- تجربة توضح الفرق قبل إدارة الموارد وبعد الإدارة

حسب المعيار العالمي للسيرفرات عالية الأداء، يجب ان يتجاوز زمن استجابة الـ API في الحالات الطبيعية 200ms وفي حالات الضغط العالي يجب ان يتجاوز 500ms لضمان تجربة مستخدم سليمة.

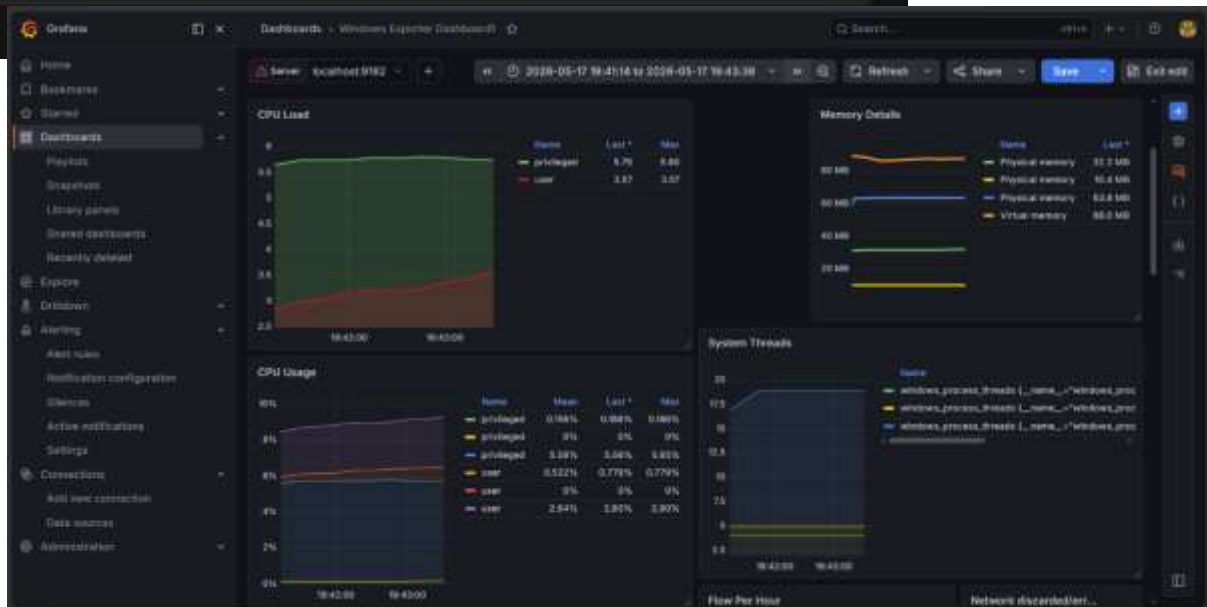
اما بالنسبة لاستهلاك المعالج فالوضع المثالي الوضع المثالي يتراوح بين ٣٠٪ الى ٧٠٪ حيث لا يجب ان يتجاوز الـ ٨٠٪

اما الذاكرة فان نسبة الاستهلاك المستمر المثلثي تتراوح بين الـ ٧٥٪ الى ٨٠٪ حيث قمنا باستخدام مكتبة locust والتي تمكنا من محاكاة سلوك الالف المستخدمين المتزامنين بكفاءة عالية وقياس مدى استجابة النظام تحت الضغوط المختلفة ومن خلال هذه الأداة استطعنا الحصول على تحليل رقمي دقيق للفرق الذي أحدثه تجمع الخيوط Thread Pool وكانت النتائج كالتالي: أولاً:

قمنا بمحاكاة ١٠٠ مستخدم يقوم بطلب عرض المنتجات في الوقت نفسه (في نفس الثانية) قبل ادارة الموارد:

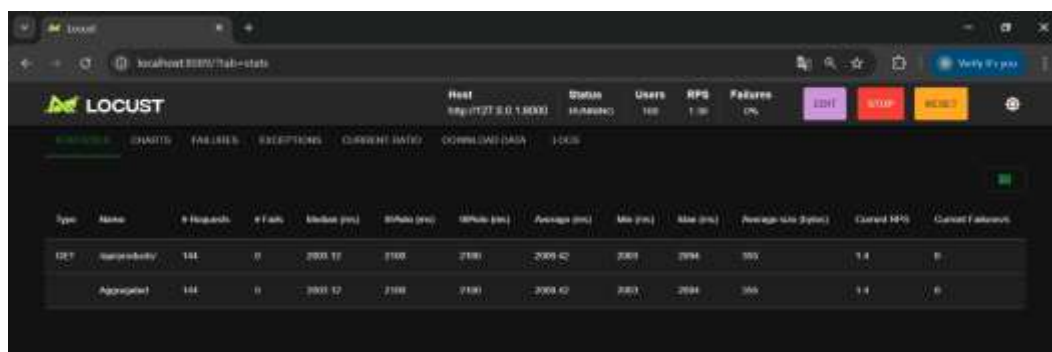


بعد إدارة الموارد:

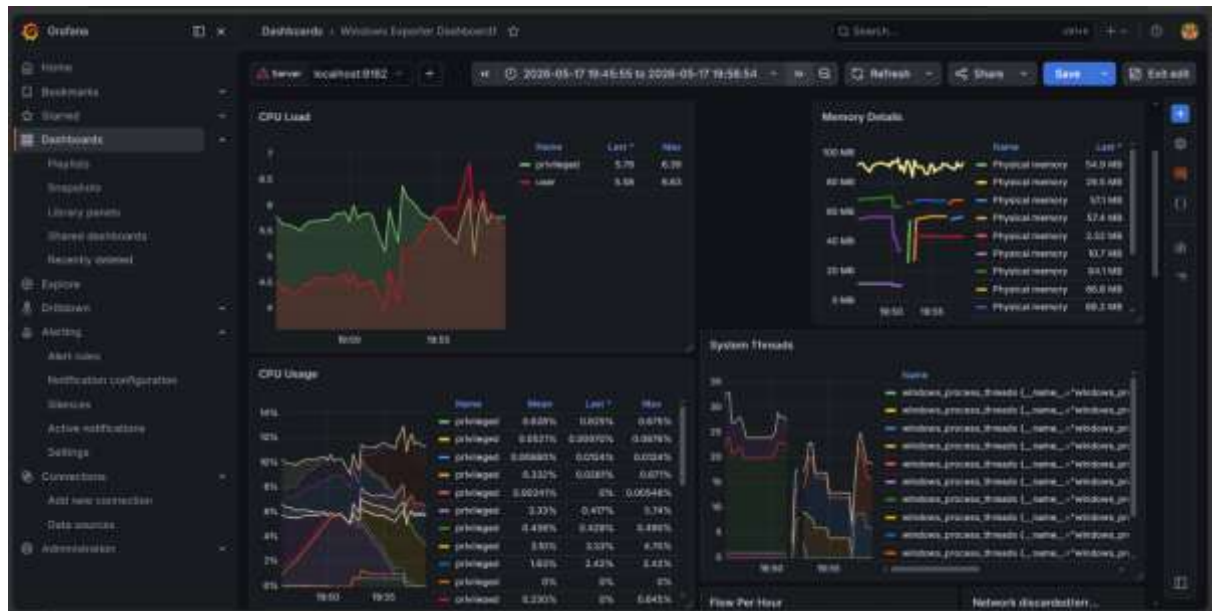


## ثانياً:

قبل إدارة الموارد:



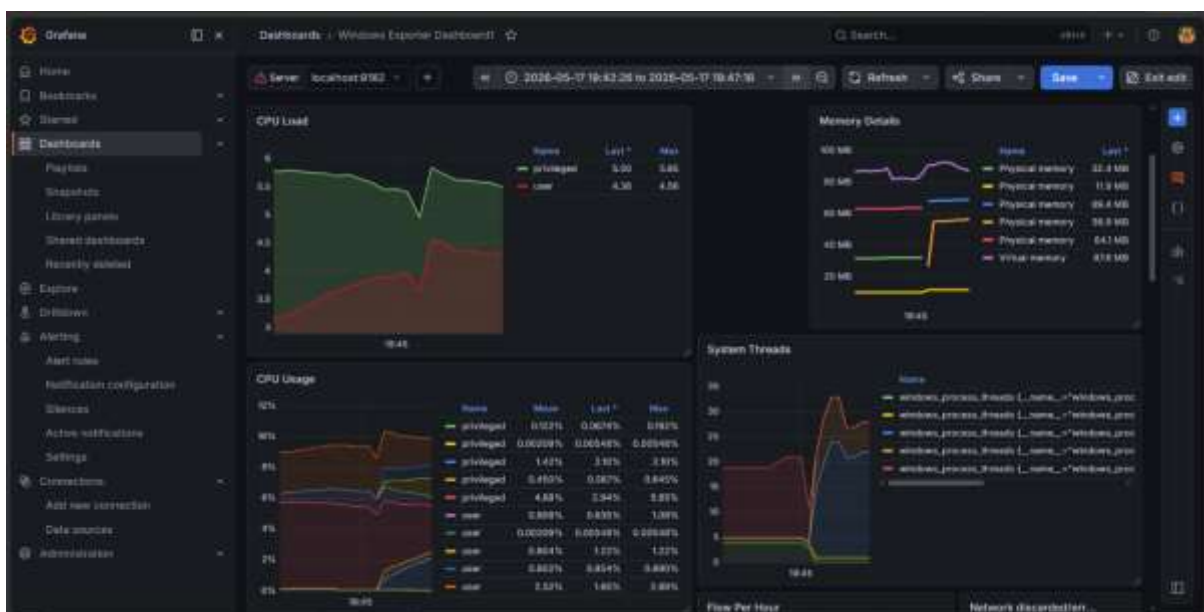




نلاحظ ان الداء اصبح سيئا جدا حيث تجاوز معدل الاستجابة 2000ms مع تخط شديد في استهلاك الذاكرة والمعالج.

بعد إدارة الموارد:

| Type       | Name                  | # Requests | # Fails | Median (ms) | 90th (ms) | 95th (ms) | Average (ms) | Min (ms) | Max (ms) | Average rate (req/s) | Current RPS | Current Failures |
|------------|-----------------------|------------|---------|-------------|-----------|-----------|--------------|----------|----------|----------------------|-------------|------------------|
| GET        | http://127.0.0.1:8000 | 373        | 0       | 3           | 9         | 14        | 14           | 2        | 136      | 306                  | 3.4         | 0                |
| Aggregated |                       | 373        | 0       | 3           | 9         | 14        | 14           | 2        | 136      | 306                  | 3.4         | 0                |



نلاحظ تحسن الاداء بشكل كبير رغم ان العمليات ثقيلة الا ان معدل الاستجابة 4ms تقريبا واستهلاك المعالج والذاكرة جيد ومستقر.

### الطلب الثالث: المعالجة غير المتزامنة Asynchronous Queue:

نقل المهام التي لا يحتاج المستخدم لانتظارها (مثل اصدار فواتير أو ارسال الاشعارات ) خارج المسار الرئيسي للطلب.

لفصل وقت الاستجابة للمستخدم عن وقت المعالجة الثقيلة أي بدلا من المعالجة المباشرة عند ارسال طلب للسيرفر ومراجعة قاعدة البيانات وتوليد فاتورة وارسل ايميل ثم الرد على المستخدم (الطلب يسير بمسار واحد ) هذا سيجعل المستخدم ان ينتظر كثير أمام الشاشة بدلا من الحصول على طلبه بسرعة، لذلك نجعل العمليات الثقيلة والتي لا يحتاج المستخدم انتظارها للعمل خارج المسار الرئيسي للطلب الأساسي، أي بمجرد استلام الطلب والتأكد من صحته يقوم السيرفر بجعل المهام الثقيلة تعمل على مسار اخر ويرسل الرد فورا للمستخدم.

وكما لاحظنا عند ارسال طلب شراء على الـ postman انتهى الطلب في أجزاء من الثانية وقام بإرسال Accepted 202.

وفي المسار الخلفي بدأ العمل بشكل مستقل لإصدار الفواتير وارسل الاشعارات .

لذلك قمنا باستخدام ThreadPoolExecutor لتنفيذ المهام الثقيلة خارج المسار الرئيسي للطلب أي لتنفيذ المهام بالخلفية (Background Tasks) لبعض العمليات التي لا يحتاج المستخدم انتظارها مثل:

- إرسال إيميل
- إصدار فاتورة

لذلك بدل من تنفيذها داخل نفس الطلب، قمنا بتشغيلها ضمن Threads منفصلة.

```
184         quantity=quantity
185
186
187
188         io_executor.submit(send_email_notification, order.id)
189
190
191
192
```

عند تنفيذ انشاء طلب شراء تم نقل مهمة إرسال الإيميل الى Thread منفصل.

```
290
291
292
293         cpu_executor.submit(generate_invoice, order.id)
294
295
296
297
```

وعند تنفيذ عملية الدفع تم نقل عملية توليد الفاتورة الى CPU Thread منفصل.

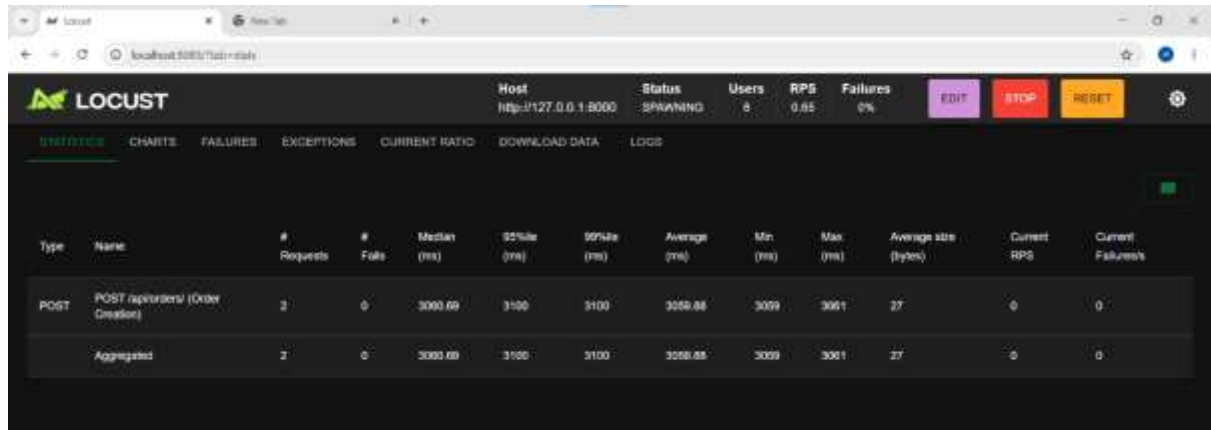
```
[12/May/2026 10:23:21] "POST /api/orders/ HTTP/1.1" 202 38
Running Email Task in thread: IO_Worker_1
Email sent successfully for order 11317
```



```
[12/May/2026 10:26:26] "POST /api/orders/11317/pay/ HTTP/1.1" 202 41
Running Invoice Task in thread: CPU_Worker_0
Invoice generated for order 11317
```

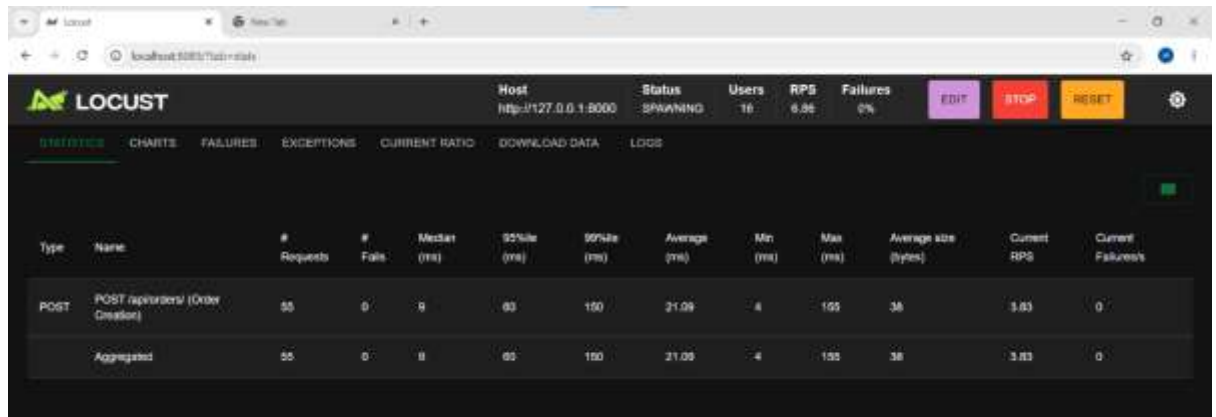
ظهر في التيرمينال أن المهام تعمل ضمن Threads مختلفة وهذا يثبت أن المعالجة تتم في الخلفية وليس داخل المسار الرئيسي للطلب.

كما قمنا بمحاكاة ٥٠ مستخدم على LOCUST وظهرت النتائج التالية :



١. تم تنفيذ جميع العمليات ضمن نفس المسار الرئيسي للطلب، أي أن السيرفر كان ينتظر انتهاء إنشاء الطلب وإصدار الفاتورة وإرسال الإيميل قبل إرسال الاستجابة للمستخدم.

وقد تم اختبار السيرفر باستخدام Locust لمحاكاة ٥٠ مستخدم متزامن فكان متوسط زمن الاستجابة يتجاوز ٣ ثواني .



٢. فصل العمليات مثل إصدار الفواتير وإرسال الإشعارات عن المسار الرئيسي للطلب باستخدام ThreadPoolExecutor، حيث أصبح السيرفر يرسل الاستجابة مباشرة للمستخدم ثم يكمل تنفيذ المهام بالخلفية ضمن Threads مستقلة.

وقد تم اختبار السيرفر باستخدام Locust لمحاكاة ٥٠ مستخدم متزامن، فكانت النتائج أفضل بشكل واضح، حيث انخفض متوسط زمن الاستجابة الى حوالي: ١١ ms بعد أن كان يتجاوز ٣ ثواني في الحالة الأولى..

## الطلب الرابع: المعالجة الدفعية المتوازية المجدولة تلقائياً ( Parallel Batch Processing & Scheduling):

### ١. الهدف من الطلب:

تحسين أداء النظام عند التعامل مع كميات ضخمة من البيانات الناتجة عن عمليات الشحن الكثيف (Bulk Charging). يهدف هذا المتطلب الى تجنب معالجة البيانات دفعة واحدة بشكل تسلسلي مما يسبب اختناقاً للمعالج (CPU Bottleneck)، واستبدال ذلك بتقطيع البيانات الى حزم صغيرة (Chunks) وتوزيعها ديناميكياً على خيوط معالجة متعددة (Parallel Workers) تعمل بالتوازي في الخلفية عبر جدول زمني تلقائي (APScheduler).

### ٢. التقنيات والأدوات المستخدمة:

- APScheduler (Advanced Python Scheduler): لجدولة المهمة وتفعيلها تلقائياً بانتظام بدون تدخل برمجي خارجي.
- ThreadPoolExecutor (cpu\_executor): مفسر الخيوط التفرعية المخصص للعمليات الحسابية الكثيفة (CPU-bound tasks).
- Django ORM Optimization (select\_related): لجلب البيانات المرتبطة من قاعدة البيانات ككتلة واحدة (Eager Loading) لتسريع الاستعلامات وضمان الأمان التفرعي (Thread Safety).

### ٣. الشرح للملف core/tasks.py:

- تم بناء الدالتين أساسيتين لتحقيق هذا المتطلب وفصل المسؤوليات أكاديمياً:
- a. Orchestrator Function (process\_daily\_sales\_chunks): الدالة المنسقة التي تقيس الأداء، تقسم البيانات، وتوزع المهام.
  - b. Orchestrator Function (process\_daily\_sales\_chunks): الدالة المنسقة التي تقيس الأداء، تقسم البيانات، وتوزع المهام.

### ٤. تحليل المخرجات وسلوك النظام بالتيرمينال:

عند تشغيل المجدول وضخ ٥٠,٠٠٠ سجل عبر الـ BulkDataChargerAPIView نلاحظ التالي:

```
APScheduler started successfully within 'core' app!
Starting Bulk Data Injection...
Successfully injected 50000 paid orders into PostgreSQL!
[18/May/2026 03:41:40] "POST /api/charge-db/ HTTP/1.1" 201 71
```

```
Processing began without partitioning all data (250000 register)...\nProcessing output without division:\n  Number of requests processed: 250000\n  Final price of orders (Total Revenue): 50000000.00\n  Total time consumed: 11.2513 s
```

```
Processing began by splitting the data into batches: 1000...
```

```
[Parallel Worker] Chunk #1 Finished Processing !\n[Parallel Worker] Chunk #2 Finished Processing !\n[Parallel Worker] Chunk #5 Finished Processing\n[Parallel Worker] Chunk #7 Finished Processing\n[Parallel Worker] Chunk #9 Finished Processing\n[Parallel Worker] Chunk #13 Finished Processing\n[Parallel Worker] Chunk #14 Finished Processing\n[Parallel Worker] Chunk #11 Finished Processing\n[Parallel Worker] Chunk #15 Finished Processing
```

```
[Parallel Worker] Chunk #161 Finished Processing !\n[Parallel Worker] Chunk #164 Finished Processing !\n[Parallel Worker] Chunk #200 Finished Processing !\n[Parallel Worker] Chunk #166 Finished Processing !\n[Parallel Worker] Chunk #162 Finished Processing !
```

```
Processing outputs with segmentation:
```

```
  Number of batches processed (Total Chunks): 250\n  Number of requests processed: 250000\n  Final price for orders: 50000000.00\n  Final time consumed: 0.8135 s
```

```
[BENCHMARK RESULT]
```

```
Batch Processing it was faster by a factor of 10.4377 s
```

```
Batch Benchmark Finished Successfully
```

- تفسير السلوك التفرعي: تظهر العبارة [Parallel Worker] Chunk #x Finished processing بشكل متداخل وغير متتابع في التيرمينال؛ مما يثبت أن خيوط المعالج تعالج مجموعات البيانات بالتوازي (Concurrent Execution).
- نتيجة اختبار الأداء: تظهر مخرجات المقارنة الزمنية تفوق الـ Parallel Batch Processing على المعالجة التقليدية بفارق زمني شاسع، مما يثبت ارتفاع معدل الإنتاجية لـ ( System Throughput) مع زيادة حجم البيانات.

## الطلب الخامس: توزيع الأحمال وتوسيع النظام (Load Balancing & Distribution):

### ١. الهدف من الطلب:

محاكاة تخديم الأنظمة الضخمة تحت ضغط الطلبات الكثيف (High Traffic Systems) من خلال تطبيق مفهوم التوسع الأفقي (Horizontal Scaling). يهدف هذا الطلب الى الغاء فكرة السيرفر الفردي ونقطة الفشل الواحدة (Single Point of Failure)، عبر تشغيل عدّة نسخ مستقلة من المشروع وتوزيع طلبات الزبائن الواردة عليها بالتوازن.

### ٢. التقنيات والأدوات المستخدمة:

- Nginx HTTP Server: يعمل كـ Reverse Proxy وموزع أحمال خارجي حقيقي (Load Balancer).
- Round Robin Algorithm: الخوارزمية السياسية المتبعة في Nginx لتوزيع الطلبات على السيرفرات بالتناوب الدائري المتساوي.
- Multi-Instance Architecture: تشغيل نسختين مستقلتين من تطبيق Django على بورتات مختلفة (٨٠٠٠ و ٨٠٠١) يتصلان بقاعدة بيانات موحدة PostgreSQL.

### ٣. إعداد خادم وموزع الأحمال (Nginx Configuration):

تم تعديل ملف الإعدادات الخاص بـ Nginx (nginx.conf) ليقوم بالاستماع على البورت الموحد ٨٠ وتميرير الطلبات تفرعياً لمجمع سيرفرات دجانغو المسمى `django_servers`.

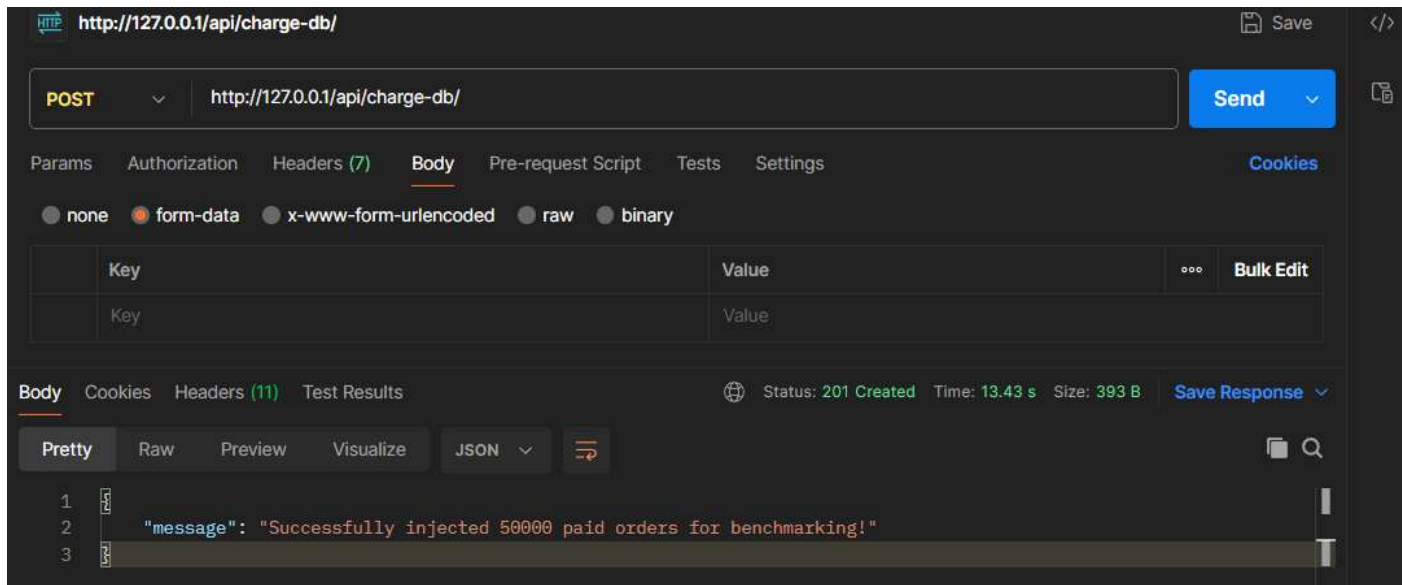
### ٤. طريقة تشغيل واختبار المنظومة الموزعة:

لإثبات تفعيل المنظومة الحقيقية على نظام التشغيل، يتم فتح ثلاث نوافذ أوامر مستقلة:

- النافذة الأولى: تشغيل السيرفر الأول: `python manage.py runserver 127.0.0.1:8000`
- النافذة الثانية: تشغيل السيرفر الثاني: `python manage.py runserver 127.0.0.1:8001`
- النافذة الثالثة: تشغيل موزع الأحمال الخارجي: `path of "nginx.exe" file`

### ٥. تحليل المخرجات من Postman وTerminal:

عند إرسال طلبات دفع متتالية من برنامج Postman الى الرابط الموحد الموجه للبورت ٨٠: GET `http://127.0.0.1/products` نلاحظ السلوك الهندسي التالي:



a. تفسير سلوك خوارزمية الـ Round Robin: عند إرسال طلبات متتالية، يستقبل Nginx الطلبات ويوجهها كالتالي:

- الطلب الأول والثالث تظهر خطوط تسجيل استلامهما (HTTP Logs) في تيرمينال السيرفر ٨٠٠٠.
- الطلب الثاني والرابع تظهر خطوط تسجيل استلامهما فوراً في تيرمينال السيرفر ٨٠٠١.

b. هذا التناوب الدائري المطلق يؤكد نجاح موازنة الأحمال وتقسيم جهد المعالجة وحقن المفهوم الحقيقي للأنظمة الموزعة (Distributed Systems).

 LOCUST

Host

http://127.0.0.1/

Status

RUNNING

Users

50

RPS

30.86

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

100%

| Type | Name                               | # Requests | # Fails | Median (ms) | 95%ile (ms) | 99%ile (ms) | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | Current RPS | Current Failures/s |
|------|------------------------------------|------------|---------|-------------|-------------|-------------|--------------|----------|----------|----------------------|-------------|--------------------|
| POST | POST /api/orders/ (Order Creation) | 6033       | 0       | 9           | 970         | 1900        | 115.89       | 2        | 3522     | 38                   | 26.6        | 0                  |
|      | Aggregated                         | 6033       | 0       | 9           | 970         | 1900        | 115.89       | 2        | 3522     | 38                   | 26.6        | 0                  |

**Total Requests per Second**

**Response Times (ms)**
